

TABLE OF CONTENTS

S.No	Topic	Page No.
1	INTRODUCTION 1.1 Objective 1.2 Modules of the Project	1
2	OVERVIEW OF TECHNOLOGIES 2.1 Features of the Front-End 2.1 Features of the Back-End	5
3	SYSTEM DESIGN 3.1 Database Design 3.2 Data Flow Diagram 3.3 Entity Relationship Diagram	9
4	SOFTWARE DESCRIPTION 4.1 HTML5 4.2 CSS3 4.3 JavaScript ES6 4.4 Custom CSS / CSS Variables 4.5 Flask Stack 4.5.1 JSON File Storage 4.5.2 Flask Framework 4.5.3 Vanilla HTML & JavaScript 4.5.4 Python Runtime 4.6 Visual Studio Code 4.7 Microsoft Edge	14
5	SYSTEM ANALYSIS 5.1 Existing System 5.2 Characteristics of Proposed System	19
6	SYSTEM SPECIFICATION 6.1 Hardware Requirements 6.2 Software Requirements	22

7	PROGRAM CODING 7.1 Source Code 7.2 Screenshots	25
8	TESTING 8.1 Software Testing 8.2 Types of Testing	61
9	CONCLUSION	64
10	REFERENCE	66

LIST OF TABLES

Table No	Table Name	Page No.
1	Front-End Technologies and Features Used	6
2	Back-End Technologies and Features Used	7
3	Hardware Requirements for Client and Server	23
4	Software Requirements	24

LIST OF FIGURES

Figure No	Figure Name	Page No.
1	Database Design	10
2	Data Flow Diagram Level – 0	11
3	Data Flow Diagram Level – 1	12
4	Entity Relationship Diagram (ER Diagram)	13

ABSTRACT

Agriculture plays a vital role in the economic development of India, and effective farm management is essential for improving productivity and profitability. However, many farmers still rely on traditional methods for maintaining records of income, expenses, equipment usage, and government schemes, which often leads to inefficiency and data loss. To overcome these challenges, the VIVASAYIN KANAKKU (Smart Farm Management System) has been developed as a web-based application to assist farmers in managing their agricultural activities in a systematic and user-friendly manner.

The proposed system provides a centralized platform where farmers can record and monitor their daily farming operations, including crop income, expenses, tractor rentals, farm equipment usage, and government scheme information. The system also offers secure user authentication and role-based access for administrators and farmers, ensuring data privacy and reliability. By digitalizing farm records, the application helps users analyze financial performance and make informed decisions.

The application is developed using modern web technologies such as HTML5, CSS3, JavaScript, and Python with the Flask framework. JSON file storage is used for data management, making the system lightweight and easy to deploy. The intuitive interface enables users with minimal technical knowledge to operate the system efficiently. Overall, the VIVASAYIN KANAKKU system enhances transparency, reduces manual workload, and improves record-keeping accuracy. It serves as a practical solution for farmers to manage their farms effectively, thereby contributing to sustainable agricultural development and improved

INTRODUCTION

1. INTRODUCTION

1.1 OBJECTIVES

Agriculture is one of the most important sectors in India, providing livelihood to a large portion of the population and contributing significantly to the national economy. In many rural areas, farmers still depend on traditional methods for maintaining records of income, expenses, crop production, and equipment usage. Manual record-keeping is time-consuming, prone to errors, and difficult to manage over long periods. Due to these limitations, farmers often face difficulties in analyzing their financial status and making informed decisions.

With the rapid growth of information technology, digital solutions have become essential for improving efficiency and transparency in various fields. The application of technology in agriculture can help farmers organize their data, reduce workload, and enhance productivity. However, many existing systems are complex and not easily accessible to farmers with limited technical knowledge. Therefore, there is a strong need for a simple, reliable, and user- friendly system that supports farmers in managing their daily agricultural activities.

VIVASAYIN KANAKKU, meaning “Farmer’s Account,” is a web-based Smart Farm Management System designed to overcome these challenges. The main objective of this project is to provide farmers with an efficient digital platform for maintaining accurate and organized records. The system enables users to store and manage information related to income, expenses, tractor rentals, equipment usage, and government schemes in a centralized manner.

The main objective of the VIVASAYIN KANAKKU – Smart Farm Management System is to provide farmers with a simple, efficient, and reliable digital platform for managing their agricultural records. The system aims to replace traditional manual record-keeping methods with a computerized system that improves accuracy and accessibility.

The specific objectives of this project are as follows:

- To develop a user-friendly web-based application for farm management.
- To maintain digital records of income, expenses, and agricultural activities.
- To help farmers track tractor rentals and equipment usage.

- To reduce paperwork and minimize data redundancy.
- To improve decision-making by providing organized and accurate information.
- To promote digital literacy among farmers.

1.2 Modules of the Project

The VIVASAYIN KANAKKU system is divided into several functional modules, each responsible for performing specific tasks. These modules work together to ensure smooth and efficient operation of the system.

1. User Management Module

This module handles user registration, login, and authentication. It ensures secure access to the system for farmers and administrators. It also manages user profiles and role-based permissions.

2. Income Management Module

This module is used to record and manage income generated from agricultural activities such as crop sales and other farm-related earnings. It helps farmers monitor their revenue over time.

3. Expense Management Module

This module allows users to maintain records of expenses related to seeds, fertilizers, labor, irrigation, transportation, and other farming activities. It helps in analyzing production costs.

4. Tractor and Equipment Management Module

This module manages information related to tractors and farm equipment, including rental details, maintenance records, and usage history. It ensures efficient utilization of resources.

5. Scheme Information Module

This module provides updated information about government schemes, subsidies, and support programs for farmers. It helps users stay informed and benefit from available opportunities.

6. Data Storage and Management Module

This module handles data storage using JSON files and ensures proper organization, retrieval, backup, and security of data.

7. Reporting and Analysis Module

This module generates summaries and reports of income, expenses, and overall farm performance. It helps farmers analyze financial status and plan future activities.

8. Administration Module

This module is used by administrators to manage users, verify data, update system information, and monitor overall system performance.

9. Dashboard Visualization Module

This module provides a graphical dashboard that displays important information such as total income, total expenses, net profit, and rental activities using charts and summaries.

OVERVIEW OF TECHNOLOGIES

2. OVERVIEW OF TECHNOLOGIES

This chapter presents a detailed survey of the technologies employed in the development of the VIVASAYIN KANAKKU Smart Farm Management System. The technologies are categorized into frontend and backend components, with an analysis of the features leveraged in this project.

2.1 Features of the Front-End

The frontend of the application is built using core web technologies: HTML5, CSS3, and JavaScript ES6+. This stack was deliberately chosen over modern frameworks like React, Vue, or Angular to keep the codebase simple, maintainable, and free from complex build toolchains.

Table 1: Front-End Technologies and Features Used

Technology	Features Used in the Project
HTML5	Semantic elements (header, main, aside, nav), form controls (number, date, datetime-local, tel), ARIA attributes for accessibility, meta viewport for responsive design, data attributes for DOM manipulation
CSS3	Flexbox and Grid layouts, CSS Custom Properties (variables), media queries at 768px/900px/1024px breakpoints, transitions and animations, box shadows, border-radius, modal overlays, toast notification animations
JavaScript ES6+	Async/await for asynchronous API calls, Fetch API for REST communication, arrow functions, template literals for dynamic HTML, destructuring, spread operator, localStorage for client-side state, addEventListener for event handling, DOM manipulation
Google Fonts	Mukta (Devanagari/Latin script), Tiro Devanagari Hindi — loaded via @import in CSS
Fetch API	All API communication via window.fetch(), JSON serialization/deserialization, error handling with try/catch, HTTP method configuration (GET, POST, PUT, DELETE)

2.1.1 Front-End Architecture

The frontend follows a multi-page application (MPA) architecture rather than a single-page application (SPA) pattern. Each functional module has its own HTML page, and navigation is handled through standard anchor tags. This approach eliminates the need for a routing library and reduces JavaScript complexity. Key architectural decisions include:

- **API-Driven Design:** All data is fetched from the Flask backend via REST API calls. The frontend contains no hardcoded data except for demo purposes.
- **Client-Side State Management:** Farmer profile information is persisted in local Storage, enabling the application to restore user context across page reloads without server-side sessions.
- **Shared Utility Module:** The `utils.js` file provides common functions including `apiFetch()`, `fmt()` for currency formatting, `fmtDate()` for date formatting, `showToast()` for notifications, and mobile sidebar toggle management.
- **Responsive Design:** The sidebar navigation collapses on mobile devices, and grid layouts adapt to different screen widths using CSS media queries.

2.2 Features of the Back-End

The backend is implemented in Python using the Flask web framework. The choice of Flask over alternatives like Django, FastAPI, or Node.js was driven by its simplicity, minimal dependencies, and suitability for a college-level project that demonstrates core web development concepts.

Table 2: Back-End Technologies and Features Used

Technology	Features Used in the Project
Python 3.10+	File I/O operations, JSON parsing and serialization, datetime processing, uuid generation for tokens, time module for expiry tracking, list comprehensions, decorators for middleware
Flask	URL routing with <code>@app.route</code> decorator, HTTP request/response handling, <code>jsonify()</code> for JSON responses, <code>send_from_directory()</code> for static file serving, <code>request.json</code> and <code>request.args</code> for input parsing
Flask-CORS	Cross-Origin Resource Sharing configuration for <code>/api/*</code> endpoints, wildcard origin support for development, credentials support
JSON Storage	File-based persistent storage for all data entities, UTF-8 encoding

2.2.2 Back-End Architecture

The backend follows a RESTful API architecture with clear separation between routing, business logic, and data access. Key architectural characteristics include:

- **Stateless Design:** No server-side sessions are maintained. The `farmer_id` is passed with each request, and admin sessions use short-lived Bearer tokens stored in `tokens.json`.
- **Data Layer Abstraction:** The `read_json()` and `write_json()` helper functions abstract the file system operations, making it easy to migrate to a database in the future.
- **Middleware Pattern:** The `require_admin()` decorator implements Bearer token authentication as middleware, applied to all admin-only endpoints.
- **Static File Serving:** Flask serves HTML pages and static assets directly, eliminating the need for a separate web server like Nginx or Apache in development.

SYSTEM DESIGN

3. SYSTEM DESIGN

3.1 DATABASE DESIGN:

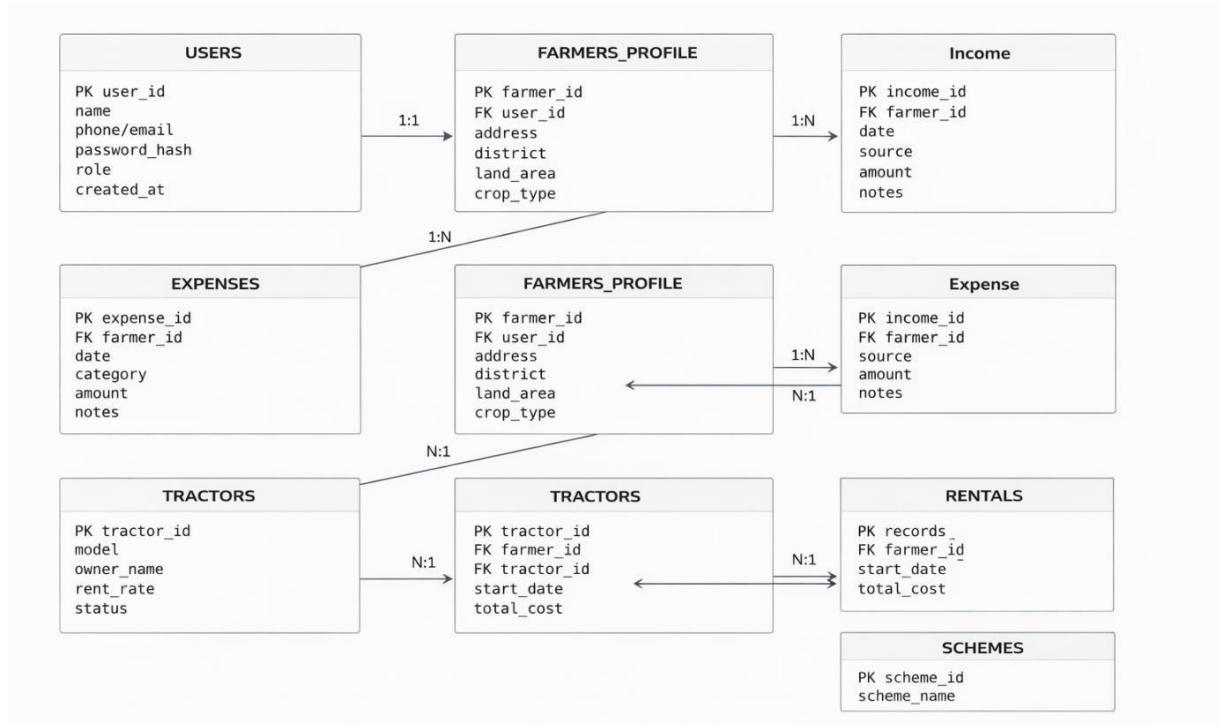


Figure 1: This diagram shows the database structure and relationships between users, farmers' profiles, income, expenses, tractors, rentals, and government schemes in the farmer management system.

The database design shows how information is stored in the system in different tables.

- The **Users** table stores login details.
- The **Farmers Profile** table stores farmer information.
- The **Income** table stores income details.
- The **Expenses** table stores expense details.
- The **Tractors** table stores tractor information.
- The **Rentals** table stores rental records.
- The **Schemes** table stores government schemes.

3.2 Data Flow Diagram (DFD)

Data Flow Diagram level -0

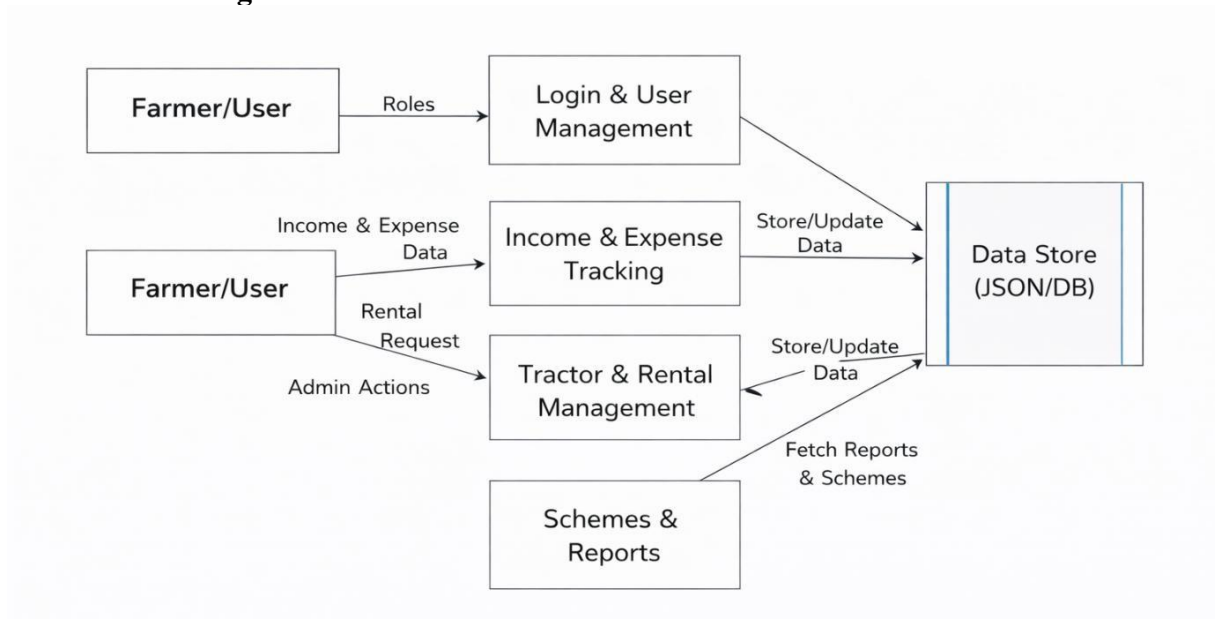


Figure 2: This diagram illustrates how the farmer/user interacts with login management, income and expense tracking, tractor rental management, and schemes reports, with all data stored and updated in the central data store.

The Data Flow Diagram shows how data moves inside the system.

- Farmers and Admin enter information into the system.
- The system processes this information.
- All data is stored in the main data storage.
- When users want reports or details, the system gets the data and shows it.
- Main processes include:
 - Login Management
 - Income and Expense Tracking
 - Tractor Rental Management

Data Flow Diagram Level - 1

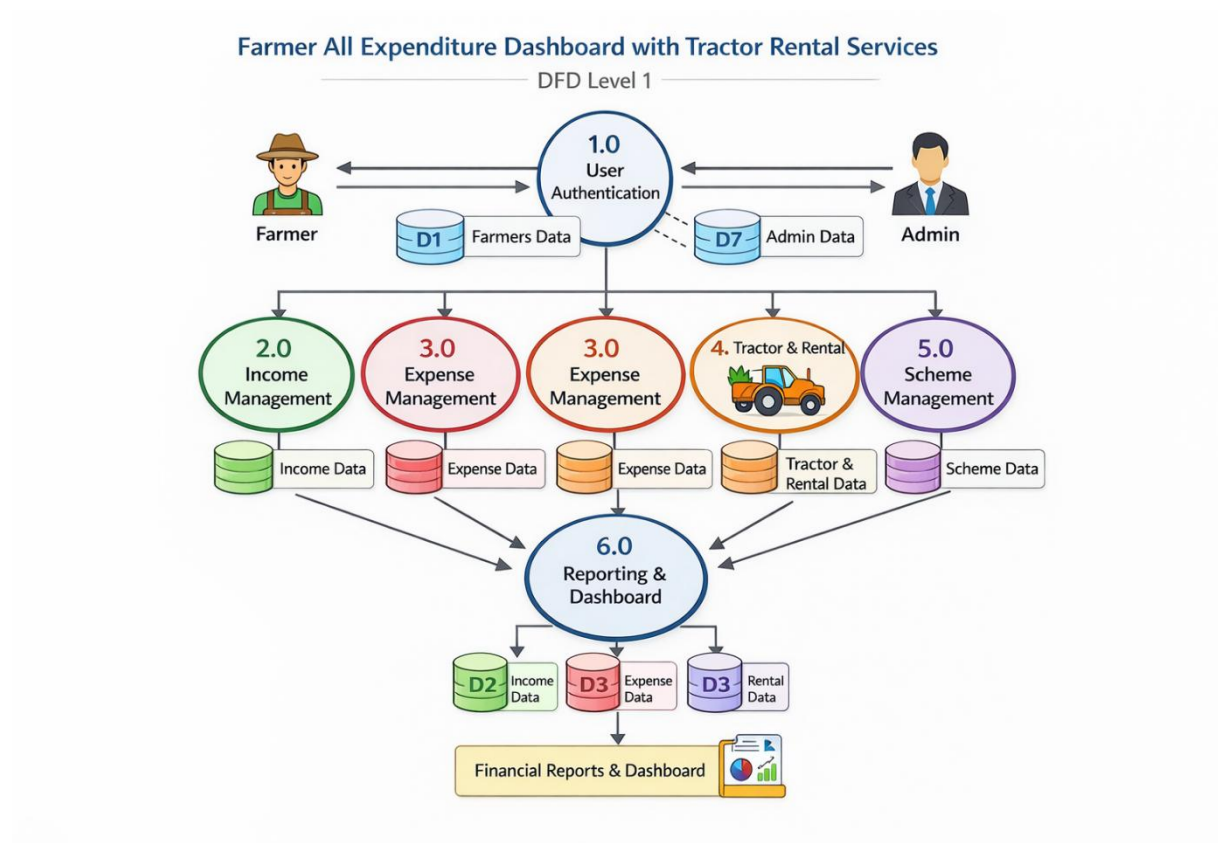


Figure level 1: This DFD Level 1 shows how farmer and admin data flow through modules like user authentication, income management, expense management, tractor rental management, and scheme management to generate financial reports and dashboards.

The DFD Level 1 of the Farmer All Expenditure Dashboard with Tractor Rental Services explains how data moves inside the system by dividing it into major processes. The two main external entities are the Farmer and the Admin. First, the User Authentication process verifies login details using stored farmer and admin data. After successful login, the farmer can access different modules such as Income Management, Expense Management, and Tractor & Rental Management, while the admin can manage government schemes through the Scheme Management process. Each process stores and retrieves information from its respective data store.

3.3 Entity Relationship Diagram (ER Diagram)

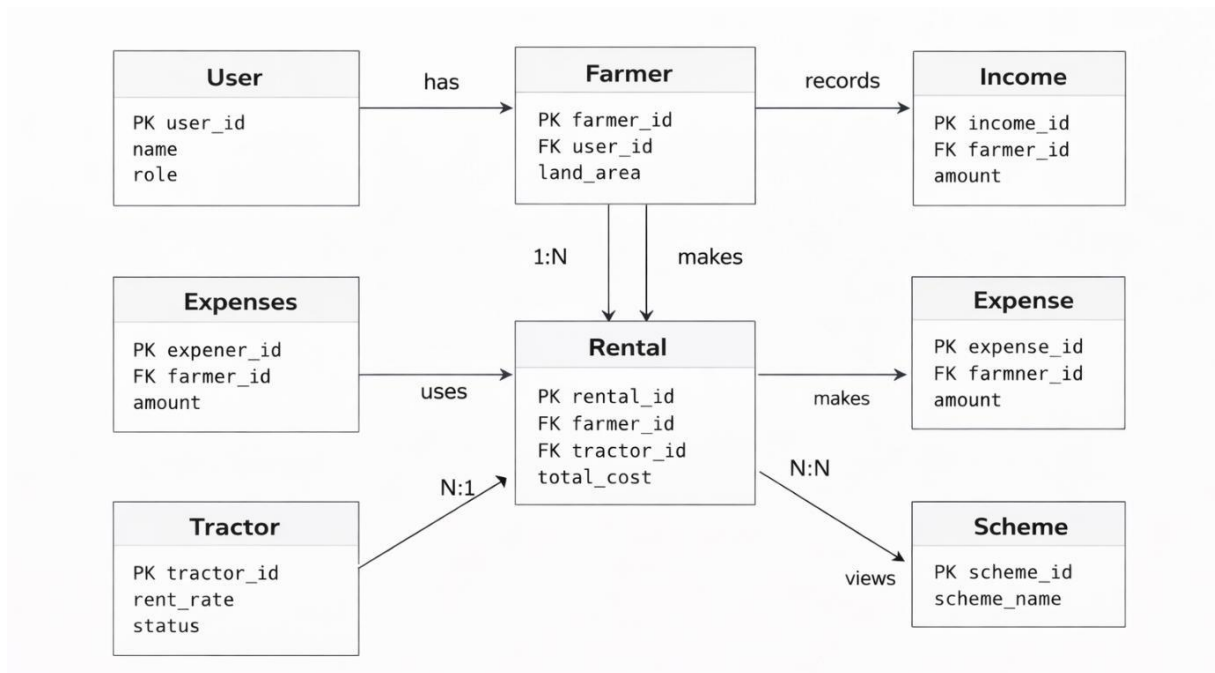


Figure 3: This ERD shows the relationships between users, farmers, income, expenses, tractor rentals, and government schemes used in the farmer expenditure and tractor rental management system.

The ER diagram shows how different parts of the system are related to each other.

- One **User** has one **Farmer Profile**.
- One **Farmer** can have many income records.
- One **Farmer** can have many expense records.
- One **Farmer** can make many rentals.
- One **Tractor** can be used in many rentals.
- Farmers can view many schemes.

SOFTWARE DESCRIPTION

4. SOFTWARE DESCRIPTION

4.1 HTML5

HTML5 (HyperText Markup Language version 5) is used to design the structure and layout of web pages. It defines the basic elements such as headings, forms, tables, images, and links.

Features:

- Supports semantic elements like <header>, <footer>, <section>, and <article>.
- Improves accessibility and readability.
- Enables multimedia support such as audio and video.
- Compatible with all modern web browsers.

HTML5 forms the foundation of the user interface and ensures proper organization of content.

4.2 CSS3

CSS3 (Cascading Style Sheets version 3) is used to control the visual appearance of web pages. It enhances the presentation by applying styles such as colors, fonts, spacing, and layouts.

Features:

- Supports responsive design using media queries.
- Enables animations and transitions.
- Improves user experience through consistent design.
- Reduces code complexity by separating style from content.

CSS3 makes the application visually attractive and easy to use.

4.3 JavaScript (ES6)

JavaScript is used to add interactivity and dynamic behavior to web pages. ES6 (ECMAScript 6) introduces modern features that simplify coding.

Features:

- Enables client-side validation.

- Improves performance with optimized syntax.
- Allows dynamic content updates without page reload.

JavaScript enhances responsiveness and user engagement.

4.4 Custom CSS / CSS Variables

Custom CSS and CSS variables are used to maintain consistent design across the application.

Features:

- Enables reuse of color schemes and layouts.
- Simplifies theme management.
- Improves maintainability.
- Supports flexible customization.

This approach helps in modifying the appearance easily without changing multiple files.

4.5 Flask Stack

Flask is a lightweight Python web framework used for developing the back-end of the application. It follows the Model-View-Controller (MVC) pattern and provides flexibility in application design.

Features:

- Simple and modular structure.
- Easy routing system.
- Built-in development server.
- Strong community support.
- Secure session handling.

Flask connects the front-end with data storage and handles business logic.

4.5.1 JSON File Storage

JSON (JavaScript Object Notation) is used for storing application data in structured format.

Features:

4.5.1.1 Easy to parse and update.

4.5.1.2 Platform-independent.

4.5.1.3 Suitable for small and medium-scale applications.

JSON storage reduces system complexity and improves portability.

4.5.2 Flask Framework

The Flask framework manages URL routing, request processing, and response generation.

Features:

4.5.2.1 Supports RESTful APIs.

4.5.2.2 Easy integration with templates.

4.5.2.3 Built-in debugging tools.

4.5.2.4 High performance for lightweight systems.

Flask ensures smooth communication between client and server.

4.5.3 Vanilla HTML & JavaScript

Vanilla HTML and JavaScript refer to the use of core web technologies without external libraries.

Features:

4.5.3.1 Reduces dependency on third-party frameworks.

4.5.3.2 Improves loading speed.

4.5.3.3 Simplifies maintenance.

4.5.3.4 Enhances security.

This approach makes the system lightweight and reliable.

4.5.4 Python Runtime

Python runtime executes server-side scripts and manages back-end operations.

Features:

4.5.4.1 Simple and readable syntax.

4.5.4.2 Strong library support.

4.5.4.3 Platform-independent.

4.5.4.4 High development speed.

Python ensures efficient processing and easy debugging.

4.6 Visual Studio Code

Visual Studio Code is used as the primary development environment.

Features:

- Intelligent code suggestions.
- Built-in debugger.
- Git integration.
- Extension support.

It improves productivity and code quality.

4.7 Microsoft Edge

Microsoft Edge is used for testing and deployment.

Features:

- High-speed performance.
- Strong security features.
- Cross-platform support.
- Developer tools for debugging.

Edge ensures proper compatibility and system reliability.

SYSTEM ANALYSIS

5. SYSTEM ANALYSIS

5.1 Existing System

The existing system refers to the traditional methods currently used by farmers to maintain farm-related records and manage daily activities. In most rural areas, farmers rely on manual record-keeping using notebooks, registers, or loose papers to document their income, expenses, crop production, equipment usage, and government scheme details.

Features of the Existing System:

1. **Manual Record Keeping**
All information is recorded manually in physical registers and notebooks, which requires significant time and effort.
2. **Paper-Based Documentation**
Data is stored in paper form, making it vulnerable to damage, loss, or misplacement.
3. **Limited Data Accessibility**
Records can be accessed only when physical documents are available, restricting portability and flexibility.
4. **High Probability of Errors**
Manual calculations and entries increase the chances of mistakes and inconsistencies.
5. **Lack of Data Security**
Physical records do not provide protection against unauthorized access.
6. **Time-Consuming Process**
Maintaining and updating records manually consumes valuable working time.
7. **No Backup Facility**
In case of fire, floods, or accidents, data may be permanently lost.
8. **Poor Reporting System**
Preparing reports requires manual compilation, which is inefficient

5.2 Characteristics of Proposed System

The proposed system, VIVASAYIN KANAKKU – Smart Farm Management System, is a web-based application designed to overcome the drawbacks of the existing system. It provides an automated and centralized platform for managing agricultural records efficiently.

Features of the Proposed System:

Digital Record Management

All farm-related information is stored in digital format, reducing dependency on paper.

1. User-Friendly Interface

The system provides simple navigation and clear instructions, making it easy for farmers to use.

2. Centralized Data Storage

All data is stored in a structured manner using JSON files, ensuring easy retrieval.

3. Secure Authentication System

User login and password protection ensure authorized access.

4. Error Reduction

Automated calculations and validations minimize data entry mistakes.

5. Easy Data Access

Users can access their records anytime and from any device with internet access.

6. Backup and Recovery Support

Digital storage allows easy backup and recovery of data.

7. Efficient Reporting System

The system generates reports automatically, saving time and effort.

8. Scalability

The modular design supports future enhancements such as database integration and mobile applications.

9. Improved Decision Making

Organized data helps farmers analyze income, expenses, and profitability.

10. Low Operational Cost

Use of open-source software reduces maintenance expenses.

11. Environment Friendly

Reduced paper usage promotes eco-friendly practices.

SYSTEM SPECIFICATION

6. SYSTEM SPECIFICATION

6.1 Hardware Requirements

The following hardware specifications define the minimum and recommended configurations for running the application both as a client user and as a development server.

Table 3: Hardware Requirements for Client and Server

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent	Intel Core i5 / AMD Ryzen 5 or higher
RAM	4 GB (2 GB minimum for basic usage)	8 GB or higher
Storage	500 MB free disk space	1 GB or more free disk space
Display	1024x768 resolution	1920x1080 or higher
Network	Internet / Local network (development)	Broadband internet connection
Input Devices	Keyboard, Mouse / Touch screen	Keyboard, Mouse, Touch screen

For server-side deployment during development, any modern computer with at least 2 GB RAM and 100 MB storage for the application and data files is sufficient. The application has been tested and verified on standard laptop and desktop configurations.

6.2 Software Requirements

The following software components are required for the development and operation of the system. All listed software components are free and open-source, reducing the total cost of ownership to zero. The following software components are required for the development and operation of the system. All listed software components are free and open-source, which helps in minimizing the overall project cost.

The system requires an operating system such as Windows 10, Windows 11, or a Linux distribution like Ubuntu for smooth execution. The backend of the application is developed using Python, while the web framework Flask is used to manage server-side operations.

Table 4: Software Requirements

Component	Requirement	Purpose
Operating System	Windows 10/11, macOS 10.14+, Linux Ubuntu 20.04+	Hosting environment
Python	Python 3.10 or higher	Backend runtime
pip	Python package installer (bundled with Python)	Package management
Flask	Latest stable version	Web framework
Flask-CORS	Latest stable version	Cross-origin resource sharing
Web Browser	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+	Frontend rendering
IDE / Editor	Visual Studio Code (recommended)	Development environment
Version Control	Git (optional)	Source code management
Virtual Environment	Python venv (recommended)	Dependency isolation

The Python packages required for the backend can be installed using the following pip command:

Pip install flask flask-cors

No additional build tools, transpilers, or package managers are required for the frontend since the project uses vanilla HTML, CSS, and JavaScript without any compilation step.

PROGRAM CODING

7. PROGRAM CODING

7.1 Source Code

Backend: app.py (Complete)

```
from flask import Flask, jsonify, request, send_from_directory, abort
from flask_cors import CORS
import json
import os
from datetime import datetime
import uuid
import time

8

9

app = Flask(__name__)
CORS(app, resources={r"/api/*": {"origins": "*"}}, supports_credentials=True)

BASE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
FRONTEND_DIR = os.path.join(BASE_DIR, 'frontend')
PAGES_DIR = os.path.join(FRONTEND_DIR, 'pages')
STATIC_DIR = FRONTEND_DIR
DATA_DIR = os.path.join(os.path.dirname(__file__), 'data')

def read_json(filename):
    path = os.path.join(DATA_DIR, filename)
```

```
with open(path, 'r', encoding='utf-8') as f:
```

```
    return json.load(f)
```

```
def write_json(filename, data):
```

```
    path = os.path.join(DATA_DIR, filename)
```

```
    with open(path, 'w', encoding='utf-8') as f:
```

```
        json.dump(data, f, indent=2, ensure_ascii=False)
```

```
def next_id(data):
```

```
    return max((item.get('id', 0) for item in data), default=0) + 1
```

```
def ensure_data_files():
```

```
    os.makedirs(DATA_DIR, exist_ok=True)
```

```
    defaults = {
```

```
        'expenses.json': [],
```

```
        'income.json': [],
```

```
        'rentals.json': [],
```

```
        'tractors.json': [
```

```
            {"id": 1, "name": "Tractor A", "rate_per_day": 1000, "status": "Available"},
```

```
            {"id": 2, "name": "Tractor B", "rate_per_day": 1200, "status": "Available"}]
```

```
    ],
```

```
        'schemes.json': [
```

```
            {"id": 1, "name": "PM-KISAN", "full_name": "Pradhan Mantri Kisan Samman Nidhi
```

```
(PM-KISAN)", "category": "Income Support", "type": "Direct Benefit", "ministry": "Ministry
```

```
of Agriculture", "benefit": "₹6000 per year", "eligibility": "Small and marginal farmers",
```

```

"documents": ["Aadhaar", "Land records"], "apply_link": "https://pmkisan.gov.in",
"description": "Income support for small and marginal farmers."},
    {"id": 2, "name": "PMFBY", "full_name": "Pradhan Mantri Fasal Bima Yojana",
"category": "Insurance", "type": "Crop Insurance", "ministry": "Ministry of Agriculture",
"benefit": "Crop insurance coverage", "eligibility": "All eligible farmers", "documents":
["Aadhaar", "Land records", "Bank details"], "apply_link": "https://pmfby.gov.in",
"description": "Crop insurance scheme."}
],
'farmers.json': [],
'admin.json': {"username": "admin", "password": "admin123"},
'tokens.json': []
}

for fn, content in defaults.items():
    path = os.path.join(DATA_DIR, fn)
    if not os.path.exists(path):
        with open(path, 'w', encoding='utf-8') as f:
            json.dump(content, f, indent=2, ensure_ascii=False)

ensure_data_files()

def load_tokens():
    try:
        return read_json('tokens.json')
    except Exception:

```

```
    return []
```

```
def save_tokens(tokens):
```

```
    write_json('tokens.json', tokens)
```

```
def create_token(username, ttl=3600):
```

```
    tokens = load_tokens()
```

```
    token = str(uuid.uuid4())
```

```
    expires = int(time.time()) + ttl
```

```
    tokens.append({'token': token, 'user': username, 'expires': expires})
```

```
    save_tokens(tokens)
```

```
    return token
```

```
def verify_token(token):
```

```
    tokens = load_tokens()
```

```
    now = int(time.time())
```

```
    for t in tokens:
```

```
        if t.get('token') == token and t.get('expires', 0) >= now:
```

```
            return True
```

```
    return False
```

```
def revoke_token(token):
```

```
tokens = load_tokens()

tokens = [t for t in tokens if t.get('token') != token]

save_tokens(tokens)
```

```
def check_admin_credentials(username, password):

    try:

        admin = read_json('admin.json')

    except Exception:

        return False

    return username == admin.get('username') and password == admin.get('password')
```

```
def require_admin(fn):

    def wrapper(*args, **kwargs):

        auth = request.headers.get('Authorization', "")

        if auth.startswith('Bearer '):

            token = auth.split(' ', 1)[1]

            if verify_token(token):

                return fn(*args, **kwargs)

            return jsonify({'error': 'Unauthorized'}), 401

    wrapper.__name__ = fn.__name__

    return wrapper
```

```
# Frontend routes
```

```

@app.route('/', methods=['GET'])

def serve_index():

    return send_from_directory(PAGES_DIR, 'index.html')


@app.route('/pages/<path:filename>', methods=['GET'])

def serve_page(filename):

    return send_from_directory(PAGES_DIR, filename)


@app.route('/static/<path:filename>', methods=['GET'])

def serve_static(filename):

    return send_from_directory(STATIC_DIR, filename)


@app.route('/<path:subpath>', methods=['GET'])

def serve_any(subpath):

    if subpath.startswith('api/'):

        abort(404)

    candidate = os.path.join(PAGES_DIR, subpath)

    if os.path.isfile(candidate):

        return send_from_directory(PAGES_DIR, subpath)

    if not subpath.endswith('.html'):

        candidate_html = os.path.join(PAGES_DIR, subpath + '.html')

        if os.path.isfile(candidate_html):

            return send_from_directory(PAGES_DIR, subpath + '.html')

```

```
candidate_static = os.path.join(STATIC_DIR, subpath)

if os.path.isfile(candidate_static):

    return send_from_directory(STATIC_DIR, subpath)

abort(404)
```

Admin

```
@app.route('/api/admin/login', methods=['POST'])

def admin_login():

    body = request.json or {}

    username = body.get('username')

    password = body.get('password')

    if not username or not password:

        return jsonify({'error': 'Missing credentials'}), 400

    if check_admin_credentials(username, password):

        token = create_token(username)

        return jsonify({'token': token})

    return jsonify({'error': 'Invalid credentials'}), 401
```

```
@app.route('/api/admin/logout', methods=['POST'])

def admin_logout():

    auth = request.headers.get('Authorization', "")

    if auth.startswith('Bearer '):

        token = auth.split(' ', 1)[1]

        revoke_token(token)
```

```
return jsonify({'ok': True})
```

```
@app.route('/api/admin/schemes', methods=['POST'])
```

```
@require_admin
```

```
def admin_create_scheme():
```

```
    data = read_json('schemes.json')
```

```
    body = request.json or {}
```

```
    body['id'] = next_id(data)
```

```
    body['documents'] = body.get('documents') or []
```

```
    data.append(body)
```

```
    write_json('schemes.json', data)
```

```
    return jsonify(body), 201
```

```
@app.route('/api/admin/schemes/<int:sid>', methods=['PUT'])
```

```
@require_admin
```

```
def admin_update_scheme(sid):
```

```
    data = read_json('schemes.json')
```

```
    body = request.json or {}
```

```
    updated = None
```

```
    for i, s in enumerate(data):
```

```
        if s.get('id') == sid:
```

```
            data[i] = {**s, **body, 'id': sid}
```

```
            updated = data[i]
```

```
            break
```

```
if updated is None:
```

```
    return jsonify({'error': 'Not found'}), 404
```

```
write_json('schemes.json', data)
```

```
return jsonify(updated)
```

```
@app.route('/api/admin/schemes/<int:sid>', methods=['DELETE'])
```

```
@require_admin
```

```
def admin_delete_scheme(sid):
```

```
    data = read_json('schemes.json')
```

```
    data = [s for s in data if s.get('id') != sid]
```

```
    write_json('schemes.json', data)
```

```
    return jsonify({'ok': True})
```

```
# Farmers
```

```
@app.route('/api/farmers/login', methods=['POST'])
```

```
def farmer_login():
```

```
    body = request.json or {}
```

```
    number = body.get('number')
```

```
    if not number:
```

```
        return jsonify({'error': 'Missing number'}), 400
```

```
    farmers = read_json('farmers.json')
```

```
    f = next((x for x in farmers if x.get('number') == number), None)
```

```
    if f:
```

```
        return jsonify(f)
```

```
return jsonify({'need_registration': True})
```

```
@app.route('/api/farmers', methods=['POST'])
```

```
def farmer_register():
```

```
    body = request.json or {}
```

```
    number = body.get('number')
```

```
    name = body.get('name')
```

```
    if not number or not name:
```

```
        return jsonify({'error': 'Missing number or name'}), 400
```

```
    farmers = read_json('farmers.json')
```

```
    if any(x.get('number') == number for x in farmers):
```

```
        f = next(x for x in farmers if x.get('number') == number)
```

```
        return jsonify(f), 200
```

```
    f = {'id': next_id(farmers), 'number': number, 'name': name}
```

```
    farmers.append(f)
```

```
    write_json('farmers.json', farmers)
```

```
    return jsonify(f), 201
```

```
@app.route('/api/farmers/<int:fid>', methods=['PUT'])
```

```
def farmer_update(fid):
```

```
    farmers = read_json('farmers.json')
```

```
    body = request.json or {}
```

```
    updated = None
```

```
    for i, f in enumerate(farmers):
```

```

    if f.get('id') == fid:

        farmers[i] = {**f, **{k: v for k, v in body.items() if k != 'id'}}

        updated = farmers[i]

        break

    if updated is None:

        return jsonify({'error': 'Not found'}), 404

    write_json('farmers.json', farmers)

    return jsonify(updated)


# Dashboard

@app.route('/api/dashboard', methods=['GET'])
def dashboard():

    expenses = read_json('expenses.json')

    income = read_json('income.json')

    rentals = read_json('rentals.json')

    farmer_id = request.args.get('farmer_id') or request.args.get('number')

    if farmer_id:

        expenses = [e for e in expenses if str(e.get('farmer_id') or e.get('farmer_number') or "") ==
str(farmer_id)]

        income = [i for i in income if str(i.get('farmer_id') or i.get('farmer_number') or "") ==
str(farmer_id)]

        rentals = [r for r in rentals if str(r.get('farmer_id') or r.get('farmer_number') or "") ==
str(farmer_id)]

        total_expense = sum(e.get('amount', 0) for e in expenses)

        total_income = sum(i.get('total', 0) for i in income)

```

```

profit = total_income - total_expense

rental_revenue = sum(r.get('paid_amount', 0) for r in rentals)

rental_pending = sum((r.get('total_amount', 0) - r.get('paid_amount', 0)) for r in rentals)

crop_stats = {}

for e in expenses:

    c = e.get('crop', '—')

    crop_stats.setdefault(c, {'expense': 0, 'income': 0})

    crop_stats[c]['expense'] += e.get('amount', 0)

for i in income:

    c = i.get('crop', '—')

    crop_stats.setdefault(c, {'expense': 0, 'income': 0})

    crop_stats[c]['income'] += i.get('total', 0)

crops_summary = [

    {'crop': k, 'expense': v['expense'], 'income': v['income'], 'profit': v['income'] -
v['expense']}

    for k, v in crop_stats.items()

]

return jsonify({

    'total_expense': total_expense,

    'total_income': total_income,

    'net_profit': profit,

    'rental_revenue': rental_revenue,

    'rental_pending': rental_pending,

    'crops_summary': crops_summary,

    'recent_expenses': expenses[-5:][::-1],

    'recent_income': income[-3:][::-1]
})

```

```
})
```

```
# Expenses
```

```
@app.route('/api/expenses', methods=['GET'])
```

```
def get_expenses():
```

```
    data = read_json('expenses.json')
```

```
    season = request.args.get('season')
```

```
    crop = request.args.get('crop')
```

```
    farmer_id = request.args.get('farmer_id') or request.args.get('number')
```

```
    if season:
```

```
        data = [e for e in data if e.get('season') == season]
```

```
    if crop:
```

```
        data = [e for e in data if e.get('crop') == crop]
```

```
    if farmer_id:
```

```
        data = [e for e in data if str(e.get('farmer_id') or e.get('farmer_number') or "") ==
```

```
str(farmer_id)]
```

```
    return jsonify(data)
```

```
@app.route('/api/expenses', methods=['POST'])
```

```
def add_expense():
```

```
    data = read_json('expenses.json')
```

```
    body = request.json or {}
```

```
    body['id'] = next_id(data)
```

```
    body['date'] = body.get('date', datetime.now().strftime('%Y-%m-%d'))
```

```
if 'farmer_id' in body or 'farmer_number' in body:
```

```
    body['farmer_id'] = body.get('farmer_id', body.get('farmer_number'))
```

```
data.append(body)
```

```
write_json('expenses.json', data)
```

```
return jsonify(body), 201
```

```
@app.route('/api/expenses/<int:eid>', methods=['DELETE'])
```

```
def delete_expense(eid):
```

```
    data = read_json('expenses.json')
```

```
    data = [e for e in data if e.get('id') != eid]
```

```
    write_json('expenses.json', data)
```

```
    return jsonify({'message': 'Deleted'})
```

```
# Income
```

```
@app.route('/api/income', methods=['GET'])
```

```
def get_income():
```

```
    data = read_json('income.json')
```

```
    farmer_id = request.args.get('farmer_id') or request.args.get('number')
```

```
    if farmer_id:
```

```
        data = [i for i in data if str(i.get('farmer_id') or i.get('farmer_number') or "") ==
```

```
str(farmer_id)]
```

```
    return jsonify(data)
```

```

@app.route('/api/income', methods=['POST'])

def add_income():

    data = read_json('income.json')

    body = request.json or {}

    body['id'] = next_id(data)

    body['total'] = body.get('quantity_kg', 0) * body.get('price_per_kg', 0)

    body['date'] = body.get('date', datetime.now().strftime('%Y-%m-%d'))

    if 'farmer_id' in body or 'farmer_number' in body:

        body['farmer_id'] = body.get('farmer_id', body.get('farmer_number'))

    data.append(body)

    write_json('income.json', data)

    return jsonify(body), 201

```

```

@app.route('/api/income/<int:iid>', methods=['DELETE'])

def delete_income(iid):

    data = read_json('income.json')

    data = [i for i in data if i.get('id') != iid]

    write_json('income.json', data)

    return jsonify({'message': 'Deleted'})

```

Schemes

```

@app.route('/api/schemes', methods=['GET'])

def get_schemes():

    data = read_json('schemes.json')

```

```

category = request.args.get('category')

stype = request.args.get('type')

if category:

    data = [s for s in data if s.get('category') == category]

if stype:

    data = [s for s in data if s.get('type') == stype]

return jsonify(data)

```

Tractors

```
@app.route('/api/tractors', methods=['GET'])
```

```
def get_tractors():
```

```

    data = read_json('tractors.json')

    status = request.args.get('status')

    if status:

        data = [t for t in data if t.get('status') == status]

    return jsonify(data)

```

```
@app.route('/api/tractors', methods=['POST'])
```

```
def add_tractor():
```

```

    data = read_json('tractors.json')

    body = request.json or {}

    body['id'] = next_id(data)

    body['status'] = 'Available'

    data.append(body)

```

```
write_json('tractors.json', data)

return jsonify(body), 201
```

```
@app.route('/api/tractors/<int:tid>', methods=['DELETE'])
```

```
def delete_tractor(tid):
```

```
    data = read_json('tractors.json')

    data = [t for t in data if t.get('id') != tid]

    write_json('tractors.json', data)

    return jsonify({'message': 'Deleted'})
```

```
# Rentals
```

```
@app.route('/api/rentals', methods=['GET'])
```

```
def get_rentals():
```

```
    return jsonify(read_json('rentals.json'))
```

```
@app.route('/api/rentals', methods=['POST'])
```

```
def add_rental():
```

```
    rentals = read_json('rentals.json')

    tractors = read_json('tractors.json')

    body = request.json or {}

    body['id'] = next_id(rentals)

    s = datetime.strptime(body['start_date'], '%Y-%m-%dT%H:%M' if 'T' in body['start_date']
else '%Y-%m-%d')
```

```

days = 0

if body.get('end_date'):

    e = datetime.strptime(body['end_date'], '%Y-%m-%dT%H:%M' if 'T' in body['end_date']
else '%Y-%m-%d')

    days = max((e - s).days, 1)

    body['days'] = days

else:

    body['end_date'] = None

    body['days'] = 0

tractor = next((t for t in tractors if t.get('id') == body.get('tractor_id')), None)

if tractor:

    body['tractor_name'] = tractor.get('name')

    if body.get('end_date'):

        body['total_amount'] = days * tractor.get('rate_per_day', 0)

    else:

        body['total_amount'] = body.get('total_amount', 0)

    for t in tractors:

        if t.get('id') == body.get('tractor_id'):

            t['status'] = 'Rented'

    write_json('tractors.json', tractors)

body['paid_amount'] = body.get('paid_amount', 0)

paid, total = body['paid_amount'], body.get('total_amount', 0)

if total == 0 or paid == 0: body['payment_status'] = 'Pending'

elif paid >= total: body['payment_status'] = 'Paid'

else: body['payment_status'] = 'Partial'

rentals.append(body)

```

```
write_json('rentals.json', rentals)
```

```
return jsonify(body), 201
```

```
@app.route('/api/rentals/<int:rid>/pay', methods=['PUT'])
```

```
def update_payment(rid):
```

```
    rentals = read_json('rentals.json')
```

```
    body = request.json or {}
```

```
    for r in rentals:
```

```
        if r.get('id') == rid:
```

```
            r['paid_amount'] = body.get('paid_amount', r.get('paid_amount', 0))
```

```
            r['payment_mode'] = body.get('payment_mode', r.get('payment_mode'))
```

```
            if r.get('paid_amount', 0) >= r.get('total_amount', 0):
```

```
                r['payment_status'] = 'Paid'
```

```
                tractors = read_json('tractors.json')
```

```
                for t in tractors:
```

```
                    if t.get('id') == r.get('tractor_id'):
```

```
                        t['status'] = 'Available'
```

```
                write_json('tractors.json', tractors)
```

```
            elif r.get('paid_amount', 0) > 0:
```

```
                r['payment_status'] = 'Partial'
```

```
            write_json('rentals.json', rentals)
```

```
            return jsonify(r)
```

```
    return jsonify({'error': 'Not found'}), 404
```

```

@app.route('/api/rentals/<int:rid>', methods=['PUT'])

def complete_rental(rid):

    rentals = read_json('rentals.json')

    tractors = read_json('tractors.json')

    body = request.json or {}

    for r in rentals:

        if r.get('id') == rid:

            if body.get('end_date'):

                r['end_date'] = body.get('end_date')

                start = datetime.fromisoformat(r.get('start_date', '').replace('Z', '+00:00')).split('T')[0]

                end =

datetime.fromisoformat(body.get('end_date', '').replace('Z', '+00:00')).split('T')[0])

                diff_seconds = (end - start).total_seconds()

                hours = max(1, round(diff_seconds / 3600))

                days = hours // 24

                extra_hours = hours % 24

                hourly_rate = 100

                daily_rate = 800

                total = days * daily_rate + extra_hours * hourly_rate

                r['total_amount'] = body.get('total_amount', total)

                r['days'] = days

            paid = r.get('paid_amount', 0)

            total = r.get('total_amount', 0)

            if paid >= total:

                r['payment_status'] = 'Paid'

            for t in tractors:

```

```

        if t.get('id') == r.get('tractor_id'):

            t['status'] = 'Available'

            write_json('tractors.json', tractors)

        elif paid > 0: r['payment_status'] = 'Partial'

        else: r['payment_status'] = 'Pending'

        write_json('rentals.json', rentals)

        return jsonify(r)

    return jsonify({'error': 'Rental not found'}), 404

```

```
@app.route('/api/rentals/<int:rid>', methods=['DELETE'])
```

```
def delete_rental(rid):
```

```

    rentals = read_json('rentals.json')

    rentals = [r for r in rentals if r.get('id') != rid]

    write_json('rentals.json', rentals)

    return jsonify({'message': 'Deleted'})

```

```
if __name__ == '__main__':
```

```
    app.run(debug=True, port=5000)
```

Frontend: login.html (Excerpt – Key Structure)

```
<!doctype html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="utf-8" />
```

```

<meta name="viewport" content="width=device-width,initial-scale=1" />

<title>Login - Vivasayin Kanakku</title>

<link rel="stylesheet" href="../css/style.css" />

</head>

<body>

  <div class="login-panel">

    <div class="card shadow">

      <div class="card-header brand">

        <div class="brand-left">

          <div class="logo-icon"> 🌾 </div>

          <div>

            <h3>Vivasayin Kanakku</h3>

            <div class="muted">Farmer login</div>

          </div>

        </div>

      </div>

    </div>

    <div class="card-body">

      <p class="lead">Sign in with your mobile number. For demo OTP use
<strong>123456</strong>.</p>

      <form id="loginForm" onsubmit="event.preventDefault(); sendOtp();">

        <label class="label">Mobile number</label>

        <input id="phone" class="input" placeholder="e.g. 9876543210" inputmode="numeric"
maxlength="10" />

        <div class="actions">

          <button class="btn btn-primary" type="submit">Send OTP</button>

        </div>

```

s

```
<div id="step-otp" class="hidden">

  <label class="label">Enter OTP</label>

  <input id="otp" class="input" placeholder="6-digit code" maxlength="6" />

  <button class="btn btn-primary" onclick="verifyOtp()">Verify OTP</button>

</div>

<div id="step-register" class="hidden">

  <label class="label">Your name</label>

  <input id="regName" class="input" placeholder="e.g. Ram Kumar" />

  <button class="btn btn-primary" onclick="registerUser()">Register &
Continue</button>

</div>

</div>

</div>

</div>

<script src="../../js/utlis.js"></script>

<script>

let pendingNumber = null;

function sendOtp(){

  const num = document.getElementById('phone').value.trim();

  if(!num) return showToast('Enter mobile number', true);

  pendingNumber = num;

  document.getElementById('loginForm').classList.add('hidden');

  document.getElementById('step-otp').classList.remove('hidden');

}

async function verifyOtp(){
```

```

const code = document.getElementById('otp').value.trim();

if(code !== '123456') { showToast('Invalid OTP', true); return; }

const res = await fetch('/api/farmers/login', { method: 'POST', headers: {'Content-
Type':'application/json'}, body: JSON.stringify({number: pendingNumber}) });

const j = await res.json();

if(res.ok && j.name){

  localStorage.setItem('farmer', JSON.stringify(j));

  setTimeout(()=> location.href = '/', 700);

} else if (j.need_registration){

  document.getElementById('step-otp').classList.add('hidden');

  document.getElementById('step-register').classList.remove('hidden');

}

}

async function registerUser(){

  const name = document.getElementById('regName').value.trim();

  if(!name) return showToast('Enter name', true);

  const r = await fetch('/api/farmers', { method:'POST', headers: {'Content-
Type':'application/json'}, body: JSON.stringify({number: pendingNumber, name}) });

  const created = await r.json();

  if(r.ok){ localStorage.setItem('farmer', JSON.stringify(created)); setTimeout(()=>
location.href = '/', 700); }

}

</script>

</body>

</html>

```

Frontend: utils.js (Complete)

```
const API = '/api';

async function apiFetch(endpoint, options = {}) {
  try {
    const res = await fetch(API + endpoint, {
      headers: { 'Content-Type': 'application/json' },
      ...options
    });
    if (!res.ok) throw new Error('API error: ' + res.status);
    return await res.json();
  } catch (e) {
    console.error(e);
    showToast('Error connecting to server. Make sure Flask is running.', true);
    return null;
  }
}

function fmt(n) {
  return '₹' + Number(n).toLocaleString('en-IN');
}

function fmtDate(d) {
  if (!d) return '-';
  const [y, m, day] = d.split('-');
  const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
```

```

    return `${day} ${months[+m-1]} ${y}`;
  }

function showToast(msg, isError = false) {
  const c = document.querySelector('.toast-container') || (() => {
    const el = document.createElement('div');
    el.className = 'toast-container';
    document.body.appendChild(el);
    return el;
  })();
  const t = document.createElement('div');
  t.className = 'toast' + (isError ? ' error' : '');
  t.textContent = msg;
  c.appendChild(t);
  setTimeout(() => t.remove(), 3500);
}

function setActive(page) {
  document.querySelectorAll('.nav-item').forEach(el => {
    el.classList.toggle('active', el.dataset.page === page);
  });
}

document.addEventListener('DOMContentLoaded', () => {
  const toggle = document.querySelector('.mobile-toggle');
  const sidebar = document.querySelector('.sidebar');

```

```

if (toggle && sidebar) {
  toggle.addEventListener('click', () => sidebar.classList.toggle('open'));
}
try {
  const farmer = JSON.parse(localStorage.getItem('farmer'));
  if (farmer && farmer.name) {
    const el = document.querySelector('.farmer-badge span');
    if (el) el.textContent = farmer.name;
  }
} catch (e) {}
});

```

Frontend: style.css (Excerpt – Key Sections)

```

@import
url('https://fonts.googleapis.com/css2?family=Mukta:wght@300;400;500;600;700&family=T
iro+Devanagari+Hindi&display=swap');

:root {
  --green-dark: #1a4d2e;
  --green-mid: #2d7a4f;
  --green-light: #4caf7d;
  --green-pale: #e8f5ee;
  --amber: #f59e0b;
  --red: #e53e3e;
  --blue: #3182ce;
  --text-dark: #1a202c;

```

```

--text-mid: #4a5568;

--border: #e2e8f0;

--card-bg: #ffffff;

--bg: #f0f4f1;

--sidebar-w: 240px;

--shadow: 0 2px 12px rgba(0,0,0,0.08);

--radius: 14px;
}

* { margin: 0; padding: 0; box-sizing: border-box; }

body {

  font-family: 'Mukta', sans-serif;

  background: var(--bg);

  color: var(--text-dark);

  display: flex;

  min-height: 100vh;

}

.sidebar {

  width: var(--sidebar-w);

  background: var(--green-dark);

  display: flex;

  flex-direction: column;

  position: fixed;

  top: 0; left: 0;

```

```
height: 100vh;  
z-index: 100;  
}
```

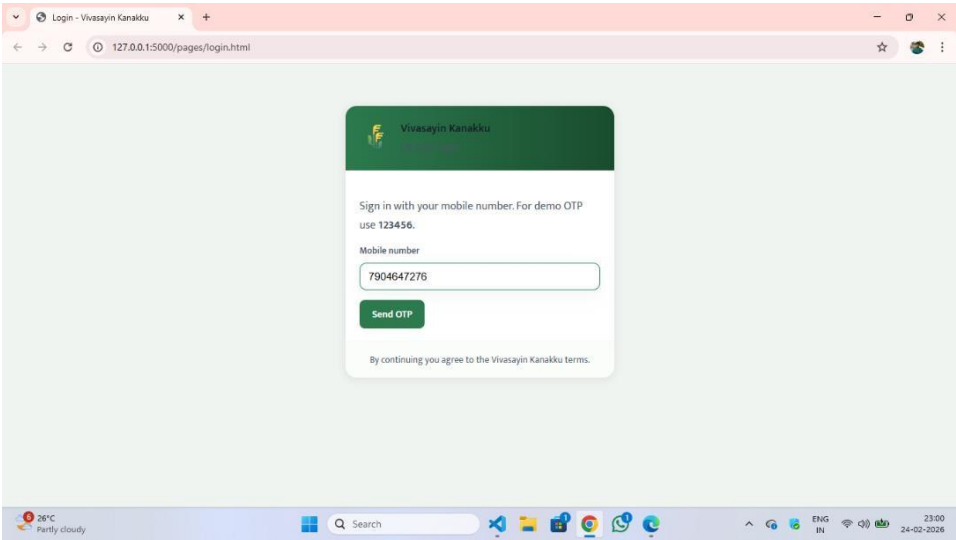
```
.stat-card {  
  background: var(--card-bg);  
  border-radius: var(--radius);  
  padding: 20px;  
  box-shadow: var(--shadow);  
}
```

```
.btn-primary { background: var(--green-mid); color: #fff; }  
.btn-outline { background: transparent; color: var(--green-mid); border: 1.5px solid var(--  
green-mid); }
```

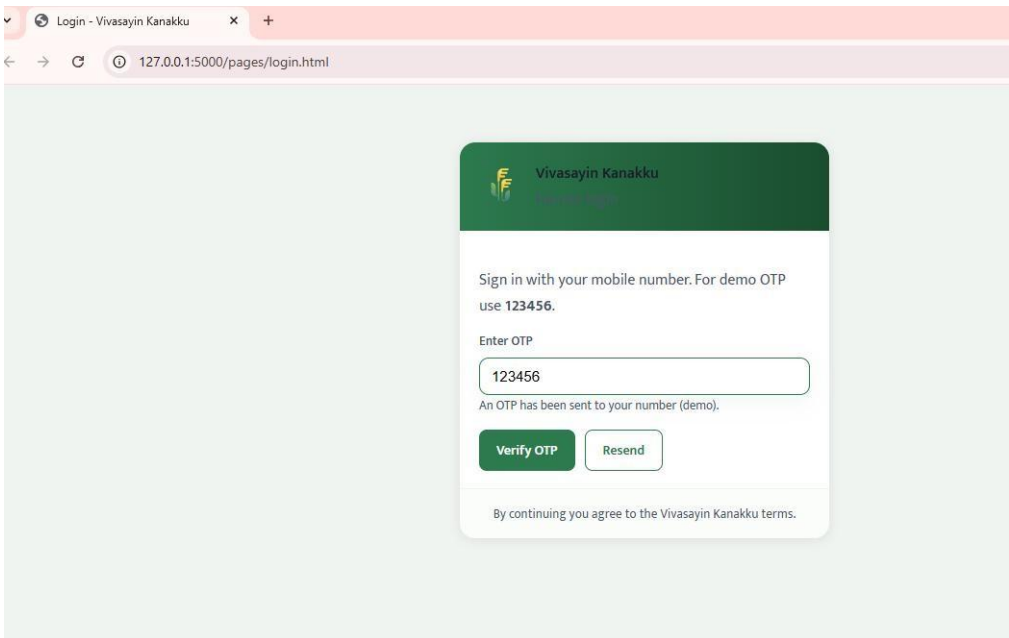
```
@media (max-width: 768px) {  
  .sidebar { transform: translateX(-100%); }  
  .sidebar.open { transform: translateX(0); }  
  .main-wrap { margin-left: 0; }  
  .mobile-toggle { display: block; }}
```

7.2 Screenshots

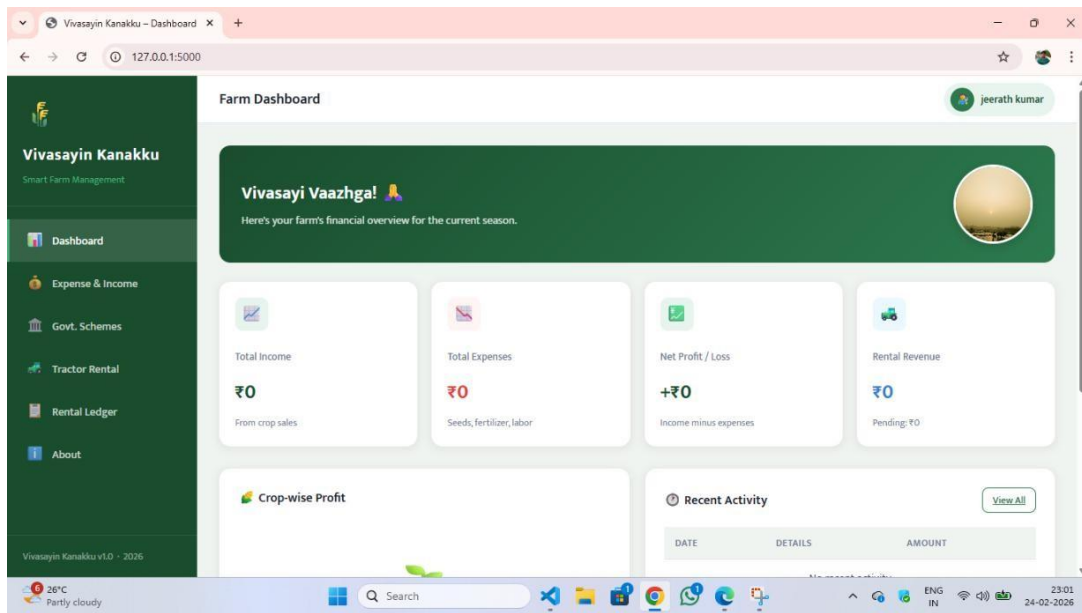
Login Page



Verify



Dashboard



Expense s Income Tracker

Vivasayin Kanakku
Smart Farm Management

Expense & Income Tracker

Add Expense

Crop Name *
e.g. Wheat, Rice, Soybean

Category *
Select or type category...

Amount (₹) *
e.g. 3500

Date
24-02-2026

Season
Winter 2026

Note
Optional note

+ Add Expense

Expense Records

[All Seasons](#) [All Crops](#)

DATE	CROP	CATEGORY	SEASON	NOTE	AMOUNT	ACTION
------	------	----------	--------	------	--------	--------

Vivasayin Kanakku v1.0

26°C Partly cloudy

23:01 24-02-2026

Income Tab

**Vivasayin Kanakku**
Smart Farm Management

Dashboard

Expense & Income

Govt. Schemes

Tractor Rental

Rental Ledger

About

Vivasayin Kanakku v1.0

Expense & Income Tracker

jeerath kumar

ExpensesIncome

Add Income (Crop Sale)

Crop Name *wheat

Quantity (kg) *2000

Price per kg (₹) *19

Date24-02-2026

SeasonWinter 2026

Sold Tomandi

Calculated Total: ₹38,000

Add Income

Income Records

DATE	CROP	QTY (KG)	RATE/KG	SEASON	SOLD TO	TOTAL	ACTION
------	------	----------	---------	--------	---------	-------	--------

Income Record

**Vivasayin Kanakku**
Smart Farm Management

Dashboard

Expense & Income

Govt. Schemes

Tractor Rental

Rental Ledger

About

Vivasayin Kanakku v1.0

Expense & Income Tracker

jeerath kumar

ExpensesIncome

Add Income (Crop Sale)

Crop Name *wheat

Quantity (kg) *e.g. 2400

Price per kg (₹) *e.g. 21.5

Date24-02-2026

SeasonWinter 2026

Sold Toe.g. Mandi, FCI, Trader

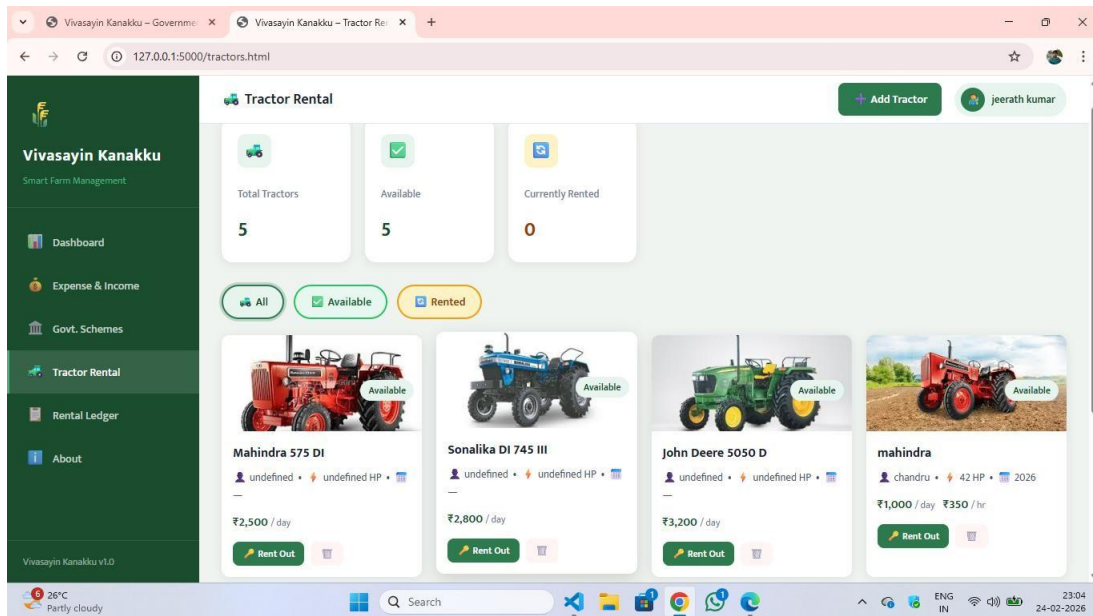
Add Income

Income Records

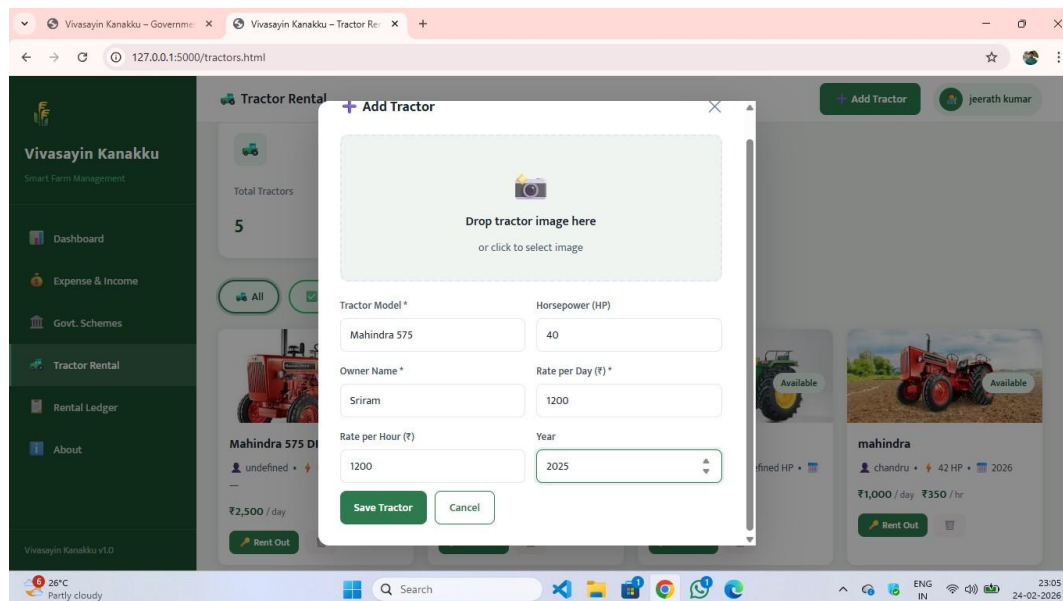
DATE	CROP	QTY (KG)	RATE/KG	SEASON	SOLD TO	TOTAL	ACTION
24 Feb 2026	wheat	2,000 kg	₹19/kg	Winter 2026	mandi	₹38,000	

Total: ₹38,000

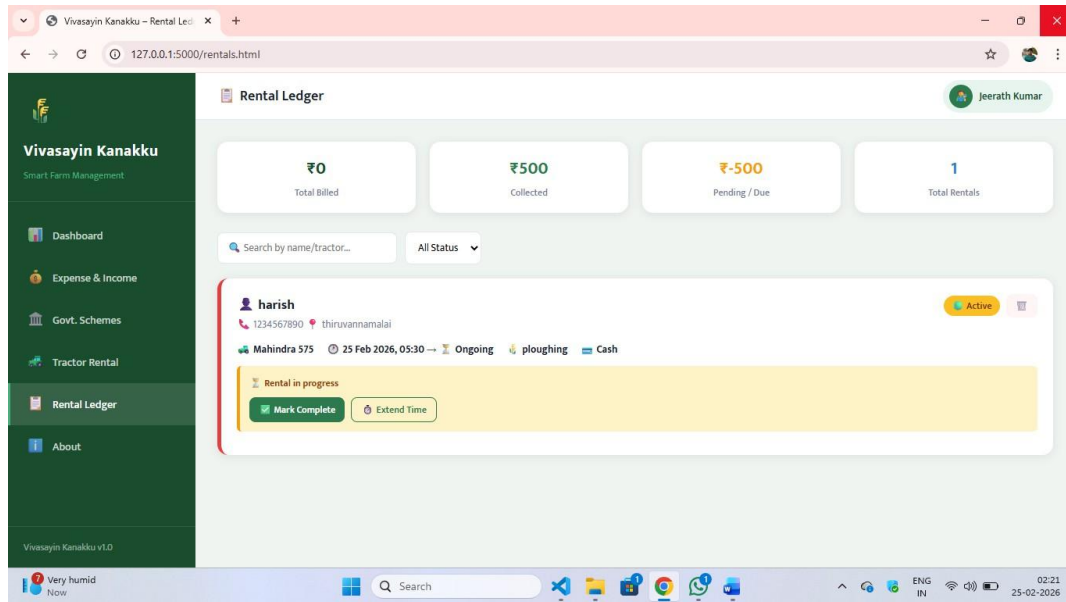
Tractor Rental



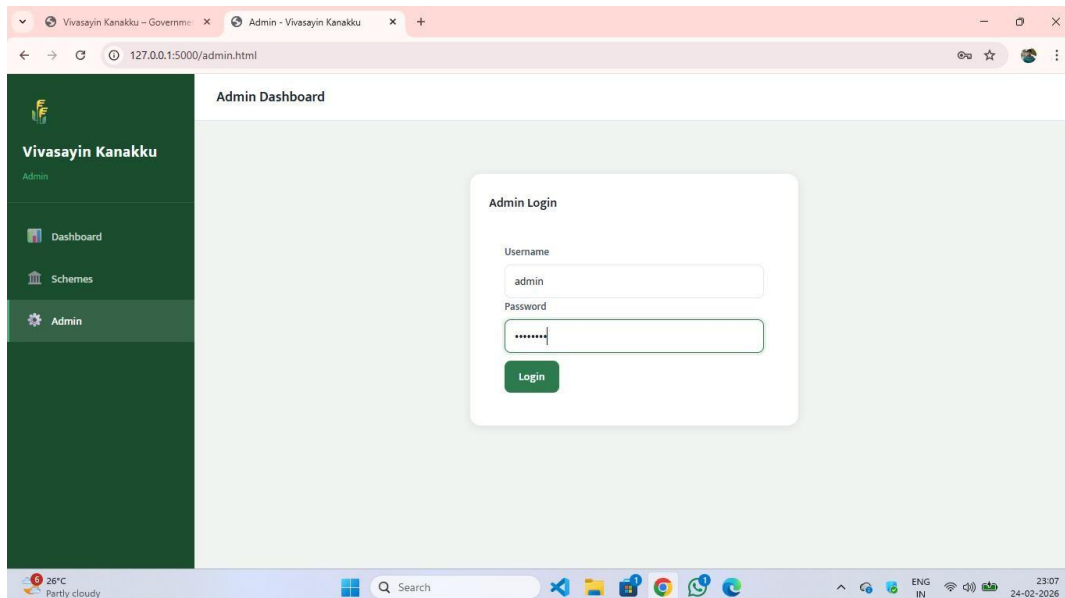
Add Tractor Modal



Rental Ledger



Admin Panel



Profile Model

The screenshot displays the Vivasayin Kanakku Farm Dashboard. A modal titled "Farmer Profile" is open, allowing the user to edit their information. The modal contains the following fields:

- Name: Jeerath Kumar
- Mobile number: 7904647276
- Email: rjeerath2005@gmail.com

At the bottom of the modal are three buttons: "Log Out" (red), "Cancel" (white), and "Save" (green). The background dashboard shows a sidebar with navigation options: Dashboard, Expense & Income, Govt. Schemes, Tractor Rental, Rental Ledger, and About. The main content area displays the user's name "jeerath kumar" and a greeting "Vivasayi Vaazhga!". It also shows financial overview cards for Total Income (₹38,000), Rental Revenue (₹0), and Pending IO. A table at the bottom lists transactions, including a wheat sale on 24 Feb 2026 for ₹38,000.

DATE	DETAILS	AMOUNT
24 Feb 2026	wheat sale - mandi	+₹38,000

TESTING

8. TESTING

8.1 Software Testing

Software testing is the process of checking whether the system functions correctly according to the requirements. It helps in finding bugs, verifying system behavior, and ensuring reliability.

In this project, testing was done to verify:

- Proper login and authentication.
- Correct data entry for income and expenses.
- Accurate rental calculations.
- Proper storage and retrieval of data.
- Correct report generation.
- Secure access for admin and farmer roles.

Testing ensures that:

- The system runs smoothly.
- There are no major errors.
- Data is stored correctly.
- Users get correct outputs.

After testing, errors were corrected and the system was re-tested to ensure proper functioning.

8.2 Types of Testing

Different types of testing were performed in this project to ensure system quality.

1. Unit Testing

Each module was tested separately.

Example: Login module, income module, rental module were tested individually.

2. Integration Testing

After testing individual modules, they were combined and tested together to check whether they work properly as a complete system.

3. System Testing

The entire application was tested as a whole to ensure that all features work correctly.

4. Functional Testing

Tested whether each function performs its expected task. Example:

- Adding income should save data properly.
- Deleting expense should remove the record correctly.

5. Validation Testing

Checked whether input fields accept correct data and reject invalid entries.

6. User Acceptance Testing (UAT)

The system was tested from a user perspective to ensure it is easy to use and meets user expectations.

7. Security Testing

Login system was tested to ensure unauthorized users cannot access data.

CONCLUSION

9. CONCLUSION

The VIVASAYIN KANAKKU – Smart Farm Management System has been successfully designed and developed to provide an efficient and user-friendly digital solution for farm management. This project aims to help farmers maintain accurate records of their agricultural activities and reduce their dependency on traditional manual methods.

The system provides a centralized platform for managing income, expenses, tractor rentals, and government scheme information. By converting manual record-keeping into a computerized system, it improves data accuracy, reduces paperwork, and saves time. The application ensures secure access through authentication and allows both farmers and administrators to manage information effectively.

The use of modern web technologies such as HTML5, CSS3, JavaScript, and Python with the Flask framework has made the system lightweight, reliable, and easy to maintain. JSON- based storage simplifies data management and enables quick retrieval of records. The responsive design ensures that the system can be accessed on different devices without difficulty.

Testing was performed at various levels to verify the correctness, security, and performance of the system. All major modules were tested successfully, and errors were identified and corrected. As a result, the system functions smoothly and meets user requirements.

In conclusion, VIVASAYIN KANAKKU is a practical and effective solution for modern farm management. It improves productivity, reduces manual effort, and contributes to sustainable agricultural development. The system can be further enhanced in the future by adding features such as mobile application support, cloud storage, advanced analytics, and integration with government portals.

Overall, this project fulfills its objectives successfully and serves as a valuable tool for farmers and administrators.

REFERENCE

10. REFERENCE

1. K. Somasundaram, *Software Engineering*, Wiley India Publications, 2019.
2. Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education, 8th Edition, 2015.
3. Python Software Foundation, "Python Documentation."
<https://docs.python.org>
4. Flask Official Documentation.
<https://flask.palletsprojects.com>
5. W3Schools, "HTML, CSS, and JavaScript Tutorials."
<https://www.w3schools.com>
6. Mozilla Developer Network (MDN), "Web Development Guides."
<https://developer.mozilla.org>
7. JSON Official Website.
<https://www.json.org>
8. Visual Studio Code Documentation.
<https://code.visualstudio.com/docs>
9. Microsoft Edge Developer Documentation.
<https://learn.microsoft.com/microsoft-edge>
10. Government of India – Ministry of Agriculture Portal.
<https://agricoop.gov.in>